Linux File System Security

Every file in Linux is owned by a user and associated group. Each **file** and **dir** in Linux system, contains a set of permissions that determines who can access them and how. You can set these permissions to limit access to these files in one of the following three ways.

1. You can restrict access to yourself alone
2. You can allow users in a pre-designated group to have access.
3. You can permit anyone on the system to have access.

A file may have read, write and execute permission. When a file is created it is automatically given some default permission. You may change these permissions to any combination you want.

User categories

Three different categories of users can have access to a file or directory. Each category can also be represented by one letter symbol.

- The owner user → **u**
- The group owner → **g**
- Others → **o**

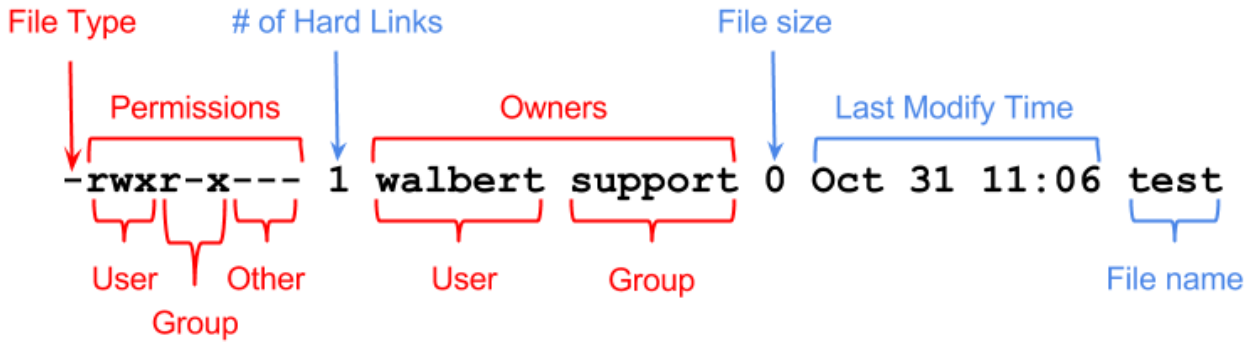
Examining File Permissions

Each of the above categories has its own set of read, write and execute permissions.

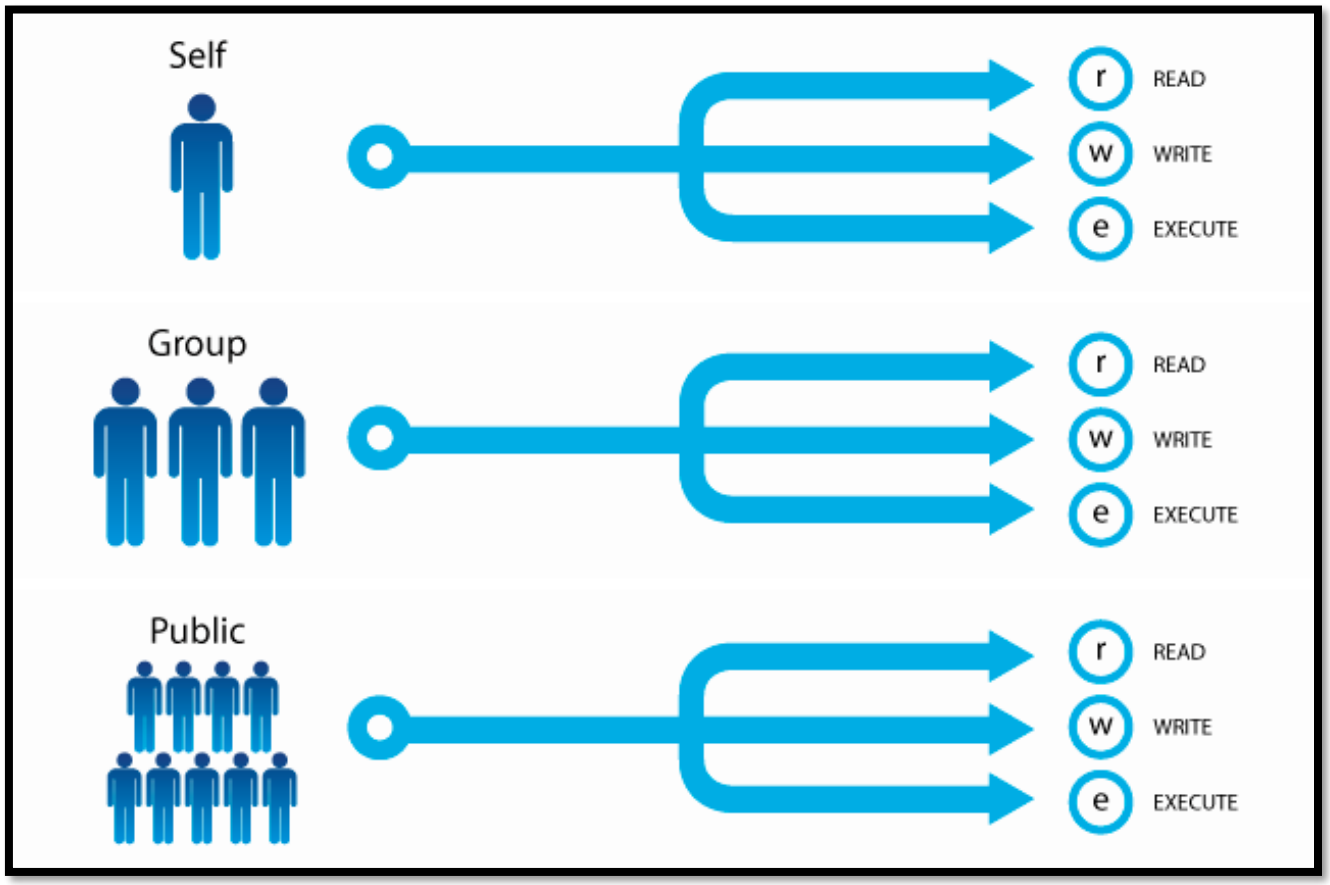
- The first set controls the users own access to its file.
- The second set controls the access of pre designated groups.
- The third set controls the access of all other users.

The three sets of **r, w, x** permissions for three categories make a total of 9 permissions. You can view these permissions by using **#ls –l** command

```
[root@yoinsights ~]#ls -l
-rwxr-x---. 1 walbert support   0 Oct 31 11:06 test
```



Whenever a user tries to access a file, first his category is identified by the Linux file system. It is 1st checked whether the user trying to access the file is falling in owner's category. If yes, the user will get user's permissions. If the user is the member of the group which is assigned as the owner group for the file, this user will come to the category of owner group and will get the groups permissions. If the user neither is owner user nor he is member of the owner group, he will fall in the third category i.e., others and will get permissions of other category.



File Permission Types

A **read** permission is represented by → **r**
A **write** permission is represented by → **w**
A **execute** permission is represented by → **x**
A **null** permission is represented by → **-**

} **Regular Permissions**

A **setuid** permission is represented by → **s**
A **setgid** permission is represented by → **S**
A **sticky bit** permission is represented by → **t**

} **Special Permissions**

Changing Access Permissions using chmod command:

```
#chmod <options> <access-permission> <file-name>
```

This command is used to change permissions of a file or directory. This is not an administrative utility i.e. any user can use this command to control the access of other users to his own files. You can use the **'chmod'** command in any of the following ways:

1. **Changing permission using symbolic method**
   - This method uses symbols to represent the category of the user and access permissions. Following lists the symbols which can be used with **chmod** command.

User category symbols	Permissions symbols	Modification instruction for chmod command
u →user	**r** → read	**+** → to add
g →group	**w** → write	**-** → to removes
o →others	**x** → execute	**=** → to replaces
a →all	**s** → setuid	
	s → setgid	
	t → sticky bit	

2. **Changing permission using absolute or numeric method**

Octal	Decimal	Permission	Representation
000	0 (0+0+0)	No Permission	---
001	1 (0+0+1)	Execute	--x
010	2 (0+2+0)	Write	-w-
011	3 (0+2+1)	Write + Execute	-wx
100	4 (4+0+0)	Read	r--
101	5 (4+0+1)	Read + Execute	r-x
110	6 (4+2+0)	Read + Write	rw-
111	7 (4+2+1)	Read + Write + Execute	rwx

The above method uses decimal numbers to represent a set of permissions. One decimal digit represents all three permissions. You can refer above table for permission mappings to number.

LAB WORK:

To Assign or change Permission

Example: 1

```
#chmod u+w-x, g+r-x, o+r project.db
```

The above command will add write permission to user category and will remove execute permission from user as well as group categories. It will add read permission to group as well as others category.

Example: 2

```
#chmod a-x, go+w project.db
```

The above remove execute permission from all categories and add write permission to group and others category.

Example: 3

```
#chmod –R 755 project
```

The above command give permission for user as rwx, group as r-x and for others r-x for entire directory and all files and dir under it.

To change user & group ownership using chown command

chown: this command is used to change the ownership of a file or directory. Only root user can execute this command.

```
Syntax: #chown <option> <new user owner> <file-name>
```

Example: 1

```
#chown ytuser project
```

This command will change the user ownership of project directory to ytuser.

Example: 2

```
#chown –R yogesh Project
```

This command will recursively change the user ownership of public directory and its contents to yogesh user.

Example: 3

```
#chown –R yogesh: tech Project
```

The above command will change the user ownership of directory Project to yogesh while group ownership to tech.

Changing group ownership using chgrp command

```
#chgrp <options> <new group owner> <file-name>
```

This command is used to change the group ownership of a file or dir. This command can only be executed by root user.

Example:

```
#chgrp mktg Project
```

This command will change the group ownership of Project directory to **mktg** group. Use **–R** to recursively change the group ownership of **Project** directory and its contents to **mktg** group.

Understanding the default file permissions and UMASK

1. When we create any file using touch, cat or VI commands they get created with default file permissions as stored in umask (User file creation mask).
2. umask is a 4 digit octal number which tells UNIX which of the three permissions are to be denied rather than granted.
3. **Umask will decide that what should be the default permissions for a file and directory when it is created.**
4. The default umask value is **0022**

```
-------------------------------------------------------
File                          Direcotry
-------------------------------------------------------
666 : Default permission      777 : Default Permission
022 : - umask value           022 : - umask value
-------------------------------------------------------
644 : final permission        755 : final permission
-------------------------------------------------------
```

To change the existing umask use the following command

```
#umask 077
```

SPECIAL PERMISSIONS OR ADVANCED PERMISSION

There are three special permissions that can be assigned to a file or directory apart from basic file permissions (rwx), they are:

1. SUID – SET USER ID
2. SGID – SET GROUP ID
3. STICKY BIT

PERMISSION	SYMBOLIC FORM	NUMERIC FORM	SYNTAX
SETUID	s OR S	4	#chmod u+s OR #chmod 4766
SETGID	s OR S	2	#chmod g+s OR #chmod 2766
STICKY BIT	t OR T	1	#chmod o+t OR #chmod 1766

Note: Where s= setuid + execute permission and S= setuid only. Same is for SGID and also for sticky bit.

SUID – SET USER ID

- When any command is executed, it creates a process in the memory. These processes are also assigned user ownerships just like the file and directories. The reason behind it is to restrict the processes which are generated by normal users. For example, if a normal user tries to execute **/sbin/init 0** command, he will get an error message that you must be superuser. This is because the process which is generated inside the system to shut it down will have the ownership of the user who has executed the command. Hence he will be denied and will get an error message.

- If **SETUID bit** is set, when the file will be executed by a user, the process will have the same rights as the owner of the file being executed. Many of the system commands are the best example for SUID, basically the owner of the commands will be root, but still a normal user can execute it.

- SETUID permissions are **set only on executable files**. Setting them on directories of text files won't have any effect.

To set and unset setuid bit on init command

```
#chmod u+s /sbin/init
#chmod u-s /sbin/init
#ls –l /sbin/init ----------To verify
```

Example

~~By default ping command is having suid, so all users can run that command but if suid is removed and a normal user wants to execute it, then it will show **'operation not permitted'.**~~

```
[root@yoinsights ~]# which ping
/bin/ping
[root@yoinsights ~]# ls -l /bin/ping
-rwsr-xr-x. 1 root root 40760 Sep 17  2013 /bin/ping
[root@yoinsights ~]#
```

~~**Note:** observe that in the permissions "–rwsr-xr-x" it contains an "s", which means SUID is placed. Let's remove suid on Ping command and logged in as normal user and check the results~~

```
[root@yoinsights ~]# chmod u-s /bin/ping
[root@yoinsights ~]# ls -l /bin/ping
-rwxr-xr-x. 1 root root 40760 Sep 17 2013 /bin/ping
[root@yoinsights ~]#

[yogesh@yoinsights ~]$ ping 192.168.189.100
ping: icmp open socket: Operation not permitted
[yogesh@yoinsights ~]$
```

SGID – SET GROUP ID

Set group ID, used on executable files to allow the file to be run as if logged into the group (like SUID but uses file group permissions). SGID can also be used on a directory so that every file created in that directory will have the directory group owner rather than the group owner of the user creating the file.

Example

When a directory is created and its group is set to some group. Now if SGID is applied to it, and the group member creates files and directory inside it, then it will get the same group rather than getting user's Primary group.

```
[root@yoinsights /]#mkdir ytdir
[root@yoinsights /]#chgrp ytgroup ytdir
[root@yoinsights /]#ls -ld ytdir
drwxr-xr-x. 2 root ytgroup 4096 Jan 6 11:05 ytdir
[root@yoinsights /]#chmod g+s ytdir
[root@yoinsights /]#ls -ld ytdir
drwxr-sr-x. 2 root ytgroup 4096 Jan 6 11:05 ytdir
[root@yoinsights /]#chmod go+w ytdir

[root@yoinsights /]#su - yogesh
[yogesh@yoinsights ~]$cd /ytdir
[yogesh@yoinsights ytdir]$touch file{1..5}
[yogesh@yoinsights ytdir]$ls -l
total 0
-rw-rw-r--. 1 yogesh ytgroup 0 Jan 6 11:07 file1
-rw-rw-r--. 1 yogesh ytgroup 0 Jan 6 11:07 file2
-rw-rw-r--. 1 yogesh ytgroup 0 Jan 6 11:07 file3
-rw-rw-r--. 1 yogesh ytgroup 0 Jan 6 11:07 file4
-rw-rw-r--. 1 yogesh ytgroup 0 Jan 6 11:07 file5
[yogesh@yoinsights ytdir]$
```

Note: when a file is created by any user it will get the group as primary group of the owner which is usually owner's private group with same name.

STICKY BIT

If sticky bit is applied on a file or directory, then only root and owner of that file or directory can delete it. Even if others are having full permissions they cannot delete the file or directory.

```
[root@yoinsights /]#chmod o+t ytdir
[root@yoinsights /]#ls -ld ytdir
drwxrwsrwt. 2 root ytgroup 4096 Jan 6 11:07 ytdir
[root@yoinsights /]#su - ytuser
[ytuser@yoinsights ~]$cd /ytdir
[ytuser@yoinsights ytdir]$ls -l
total 0
-rw-rw-r--. 1 yogesh ytgroup 0 Jan 6 11:07 file1
-rw-rw-r--. 1 yogesh ytgroup 0 Jan 6 11:07 file2
-rw-rw-r--. 1 yogesh ytgroup 0 Jan 6 11:07 file3
-rw-rw-r--. 1 yogesh ytgroup 0 Jan 6 11:07 file4
-rw-rw-r--. 1 yogesh ytgroup 0 Jan 6 11:07 file5
[ytuser@yoinsights ytdir]$rm -rf file*
rm: cannot remove `file1': Operation not permitted
rm: cannot remove `file2': Operation not permitted
rm: cannot remove `file3': Operation not permitted
rm: cannot remove `file4': Operation not permitted
rm: cannot remove `file5': Operation not permitted
[ytuser@yoinsights ytdir]$
```

ACCESS CONTROL LIST (ACL)

- Define more fine-grained discretionary access rights for files and directories.
- Often, you want to share files among certain groups and specific users. It is a good practice to designate a directory for that purpose. You want to allow those groups and users to read, and write files in that directory, as well as create new files into the directory. Such special permissions can be given using ACL.
- ACL can be applied on ACL enabled partition that means you need to enable ACL while mounting the partition.

Steps to implement ACL:

1. Create a partition and format it with ext4 file system
2. Mount a file system with ACL
3. Apply ACL on it.

To use ACL first you need to enable the ACL on the file system. Execute following to enable ACL on / file system.

```
#mount –o remount,acl /
```

To make the ACL permanent, add the ACL flag in **/etc/fstab** file as follow:

```
LABEL=/                 /                   auto    defaults,acl    1    1
```

Once you enable ACL, verify it by using **mount** command.

To apply or setting the acl use below syntax

```
#setfacl <option> < argument > < file or directory name >
```

The options are, **-m** modifies an ACL, **-x** removes ACL, **-R** Recursive into subdirectories
The possible arguments are **u:** user, **g:** group and **o:** others

Note: Whatever ACL permissions assigned to a user or group or others, it will be treated as Normal Permissions minus ACL

To assign read and execute permission to a particular user the syntax could be

```
#setfacl –m u:  <username>: <permissions> <file or dir name>
```

To check the acl permission syntax is

```
#getfacl <option> <dir/file name>
```

Options:
-d displays the default ACL
-R Recursive into subdirectories

Note: Observe that when you check for the permissions it will show a **+ sign** after normal permission, that indicate that ACL is applied on this directory.

LAB WORK

To assign read write and execute permission to a particular group

```
#setfacl –m g:<group name>:<permissions> <file or directory name>
#setfacl –m g:ytgroup:rwx  /ytdir
```

Assigning read and execute permission for a user and a group at same time.

```
#setfacl –m u:ktuser:rx,g:ktgroup:rx /ytdir
```

Removing acl for a particular user:

```
#setfacl –x u:<username> <dir name>
#setfacl –x u:ytuser /ytdir
```

Removing acl for a particular group:

```
#setfacl –x g:<group name> <directory name>
#setfacl –x g: ytgroup /ktdir
```

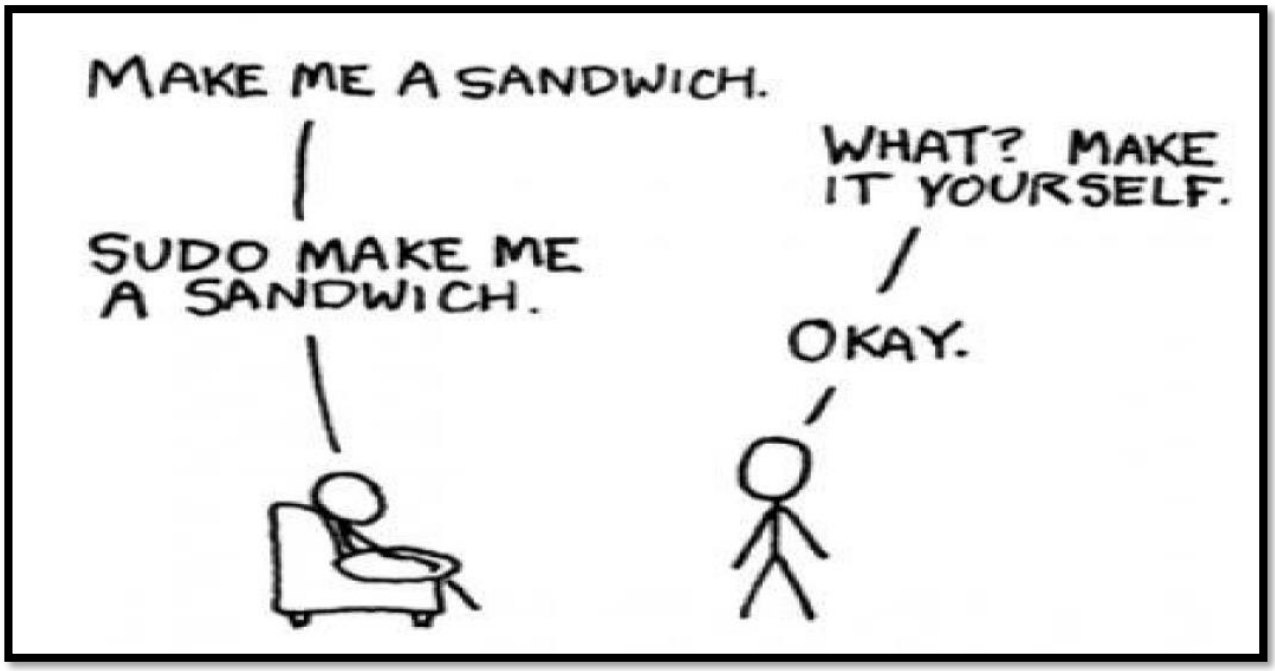
Removing all ACL permissions from a file or directory

```
#setfacl –b <dir name>
#setfacl –b /ktdir
```

ENHANCED USER SECURITY WITH SUDO

SUDO

- Sudo stands for either "substitute user do" or "super user do" (depending upon how you want to look at it). What sudo does is incredibly important and crucial to many Linux distributions. Effectively, sudo allows a user to run a program as another user (most often the root user). There are many that think sudo is the best way to achieve "best practice security" on Linux
- Users can login using their username and password and can issue administrative commands placing **sudo** in front of the commands, e.g. sudo rpm -Uvh *.rpm , to run the command which installs and updates programs in Linux (rpm).



- The file **/etc/sudoers** file has the rules that users have to follow when using sudo command. That means that whatever commands access is provided to any user in /etc/sudoers file, that user can only run those commands.
- Do not edit the **/etc/sudoers** directly; instead use **"visudo"** command to edit the sudoers file. There are two reasons for that- it prevents two users from editing the file at the same time, and it also provides limited syntax checking. Even if you are the only root user, you need the syntax checking, so use "visudo".

Advantages of using SUDO:

1. **Limited user privileges**

As we have studied above that we can restrict users to use certain commands as a privileged user as per the role of the user.
E.g.: Networking commands for Network user and Admin commands for Admin users etc.

2. **Logs of the actions done by users**

All commands executed by sudo users will be stored in **/var/log/secure** file, but still if you want you can make your own log file by passing an entry in **/etc/sudoers** file at the bottom as **"Defaults logfile=/var/log/sudo.log"** or whatever name you want, to save the logs of what commands is executed by which sudo user.

The /etc/sudoers FILE

- As we learnt above that it is the configuration file for sudo users, which is used to assign specific commands to the specific users or groups.
- Always use **visudo** command to edit this file. It prevents two users from editing the file at the same time, and it also provides limited syntax checking.
- When you run **visudo** command the output will be as follows

```
## Next comes the main part: which users can run what software on
## which machines (the sudoers file can be shared between multiple
## systems).
## Syntax:
##
##      user    MACHINE=COMMANDS
##
## The COMMANDS section may have other options added to it.
##
## Allow root to run any commands anywhere
root    ALL=(ALL)       ALL
```

- As you can see there is basically one line

root ALL=(ALL) ALL

- This lines means that the user root can execute from ALL terminals, acting as ALL (any) users, and run ALL (any) command.
- So the first part is the **user**, the second is the **terminal** from where the user can use sudo, the third is **as which user he may act**, and the last one, is which **commands** he may run.
- The advantage of **visudo** command, while editing if there are any syntax error it will be reflected as follows

```
[root@yoinsights /]#visudo
visudo: >>> /etc/sudoers: syntax error near line 99 <<<
What now?
```

LAB WORK

Lab1: Allow a user "ytuser" all privileges like root

Step: 1 First try to run fdisk command normally and see what happens. Normally, it will not allow a normal user to run privileged user's command.
Step: 2 to assign root privileges to user add a line by using sudoers file as shown below.

#visudo (save the sudoers file as we save a vim file using "**wq!**")

```
ytuser                ALL=(ALL)             ALL
```

Step: 3 now logged in as **ytuser** and run admin commands suing **sudo** before command as shown.

```
#sudo fdisk -l (or) #sudo fdisk /dev/sda
```

Note: Only for the first time of the session it will prompt for user's password to continue, but for rest of the process it will continue normally without prompting for password.

Lab2: Allow a Group "ytgroup" all privileges like root

Step: 1 to assign root privileges to group add a line by using sudoers file as shown below.

#visudo (save the sudoers file as we save a vim file using "**wq!**")

```
## Allows people in group wheel to run all commands
# %wheel        ALL=(ALL)       ALL
# %ytgroup      ALL=(ALL)       ALL
```

Step: 2 now, login as one of the user of **ytgroup** try root commands.

Lab3: Restrict a user "ytuser" to run only two root commands.

This task is very simple, just modify the previous permissions assign to ytuser. Let's give ytuser to run only **#fdisk** and **#parted** command access.

```
## The COMMANDS section may have other options added to it.
##
## Allow root to run any commands anywhere
root             ALL=(ALL)       ALL
ytuser           ALL=(ALL)       /sbin/fdisk, /sbin/parted
```

Lab4: Allow a group "ytgroup" to run only network related commands as sudo user

Step: 1 to allow a group run only network commands, first uncomment the following line as shown

```
## Networking  ---------Before Editing
# Cmnd_Alias NETWORKING = /sbin/route, /sbin/ifconfig, /bin/ping,
/sbin/dhclient, /usr/bin/net, /sbin/iptables, /usr/bin/rfcomm, /usr/bin/wvdial,
/sbin/iwconfig, /sbin/mii-tool

## Networking ---------After Editing
Cmnd_Alias NETWORKING = /sbin/route, /sbin/ifconfig, /bin/ping, /sbin/dhclient,
/usr/bin/net, /sbin/iptables, /usr/bin/rfcomm, /usr/bin/wvdial, /sbin/iwconfig,
/sbin/mii-tool
```

Observe that we have just remove **'#'** before the line to make the line readable. And also observe that it contains all networking commands.

Step: 2 just replace **"ALL"** with **"NETWORKIG"** from the last field of ytgroup line as shown.

```
## Allows people in group wheel to run all commands
# %wheel        ALL=(ALL)       ALL
# %ytgroup      ALL=(ALL)       NETWORKING
```

Step: 3 now login as one of the member of **ytgroup** and try some commands assigned it.

Lab5: Create a customize commands alias and assign it to ytgroup with network command.

Step: 1 Okay, first we need to create an alias say **"CUSTOM"** with some commands and assign it to ytgroup in addition to **NETWORK** commands and Assign it to ytgroup and save the file as shown below.

```
## Networking
Cmnd_Alias NETWORKING = /sbin/route, /sbin/ifconfig, /bin/ping, /sbin/dhclient,
/usr/bin/net, /sbin/iptables, /usr/bin/rfcomm, /usr/bin/wvdial, /sbin/iwconfig,
/sbin/mii-tool

##CUSTOM
Cmnd_Alias CUSTOM = /sbin/service, /bin/rpm, /usr/bin/yum

## Allows people in group wheel to run all commands
# %wheel        ALL=(ALL)       ALL
# %ytgroup      ALL=(ALL)       NETWORKING, CUSTOM
```

Step: 2 Login as one of the users in **ytgroup** and try newly added commands

Lab6: Create a user alias and add some users from different groups and assign some root privileges

Step1: This is very much similar to the previous task, instead of command alias we need to combine some user and give them some alias name which act as one user, but actually there are some users inside it. Then we can assign some privileges to them.

Step2: Search inside sudoers file for "**User Aliases",** under that create your own alias user as shown.

```
#visudo
## User Aliases
## These aren't often necessary, as you can use regular groups
## (ie, from files, LDAP, NIS, etc) in this file - just use %groupname
## rather than USERALIAS
# User_Alias ADMINS = jsmith, mikem
User_Alias YTADMIN = yogesh, ytuser, amit
```

Step3: let's assign some command to this alias user called **YTADMIN** and save the file.

```
## The COMMANDS section may have other options added to it.
##
## Allow root to run any commands anywhere
root            ALL=(ALL)       ALL
ytuser          ALL=(ALL)       /sbin/fdisk, /sbin/parted
YTADMIN         ALL=(ALL)       CUSTOM, /sbin/parted
```

Step4: Now login as one of those users and check the assigned commands for them

Note: Checkout sudoers file for more option and try it out on your own!!!!

Rajesh Patel
Digitally signed by Rajesh Patel
DN: cn=Rajesh Patel, c=IN, o=Yoinsights Technologies Pvt. Ltd, ou=IT Education, email=rmraj1508@outlook.com
Location: Surat
Date: 2023.06.29 08:38:20 +05'30'